Physical AI Learning Manual

Lesson 02 — How Robots Represent Their Environment

Status: Conceptual exercises completed; Isaac Sim coordinate-positioning exercise reported completed and saved by learner.
Learning stage: Module 1 — Foundations
Record date: 2026-10-12
Prerequisite: Lesson 01 — observations, policies, actions, and feedback.

> **Evidence policy:** This manual records the learner's responses and confirmations in the tutoring conversation. The original Isaac Sim USD scene and a screenshot have not been provided for independent inspection. A saved-file claim is therefore *user-confirmed*, not independently verified.

1. Goals and practical relevance

By the end of this lesson the learner should be able to: (1) describe a location with X and Y coordinates; (2) add Z to describe 3D location; (3) distinguish the world origin from a robot origin; and (4) use Isaac Sim's Transform / Translate fields to position a cube. This is a prerequisite for robot perception, kinematics, gripper targets, and coordinate transformations.

2. Concept 1 — Positions in 2D

In the teaching examples, **positive X points right** and **positive Y points up** (or forward on a tabletop). A point is written as **P = (x, y)**. The origin is (0, 0). For a displacement **ΔP = (Δx, Δy)**, the updated position is **P_new = P_old + ΔP**.

Worked example and learner answer: A cube starts at (2, 3), moves 2 units right and 1 unit downward, and arrives at **(2+2, 3−1) = (4, 2)**. The learner supplied this calculation correctly.

3. Concept 2 — Positions in 3D

A 3D point is **P = (x, y, z)**. For our exercise: X = left/right; Y = backward/forward; Z = down/up, with positive Z upward. Real coordinate frames can have different axis orientations, so always state the convention.

Worked example and learner answer: Start at (2, 3, 1); move right 1, backward 2, and upward 3. Then **P_new = (2+1, 3−2, 1+3) = (3, 1, 4)**. The learner supplied the correct answer and reasoning.

4. Concept 3 — Reference frames

A coordinate **frame** consists of an origin and oriented axes. The same physical cube can have different coordinate numbers in different frames even when the cube does not move.

When two frames' axes are **aligned (no relative rotation)**, a position relative to the robot is simply:

P_cube_in_robot = P_cube_in_world − P_robot_in_world.

Original frame example, corrected: Cube world position (6, 4), robot world origin (3, 2): cube relative to robot **(3, 2)**. An earlier illustration mistakenly labeled the robot origin as (3, 1); that label was corrected during the lesson. Had the robot actually been at (3, 1), the relative coordinates would instead be (3, 3).

Practice and learner answer: Cube world position (8, 5), robot world position (3, 2). The learner answered **(5, 3)** correctly.

Critical limitation: If the robot's axes are rotated relative to the world, subtraction alone does not perform the full frame transformation. Rotation will be covered in a later lesson.

5. Concept 4 — Hands-on in Isaac Sim 6.1

Environment: Standalone Isaac Sim 6.1 on Windows. This exercise was done in the graphical interface, not Isaac Lab and not the ROS 2 terminal.

Recorded procedure:

1. Launch Isaac Sim 6.1, create a new stage, and add a cube using the Create / Shape(s) / Cube menu.
2. Select the cube in the Stage panel, find Property → Transform → Translate.
3. Set first position **P1 = (2.0, 1.0, 1.5)**.
4. Change to second position **P2 = (3.0, 2.0, 2.5)**.
5. Compute displacement **ΔP = P2 − P1 = (1.0, 1.0, 1.0)**.
6. Save as **lesson_02_coordinates.usd**.

Learner-confirmed outcomes: The cube moved in the viewport after the first change; the learner subsequently confirmed the second position and confirmed saving **lesson_02_coordinates.usd**. The exact local location of this file was not supplied and the USD asset is **not included in this package**.

Technical note: USD Translate values are local to the prim's parent frame. They match world coordinates only under appropriate scene hierarchy / parent transform assumptions. The activity focused on straightforward positioning, not physics, coordinate rotation, or calibration.

6. Compact formulas

- 2D point: **P = (x, y)**
- 3D point: **P = (x, y, z)**
- Displacement: **ΔP = P2 − P1**
- New position: **P_new = P_old + ΔP**
- Aligned-frame relative position: **P_object/robot = P_object/world − P_robot/world**

7. Mistakes and corrections worth remembering

- **Incorrect diagram label:** A previous teaching diagram labeled robot (3, 1) while explaining a (3, 2) robot origin. Correct consistent example: world cube (6, 4); world robot (3, 2); relative cube (3, 2).
- **Coordinates require a frame:** Never interpret a position without knowing the origin and axis directions.
- **Local vs. world transforms:** Isaac Sim's Translate fields should not automatically be assumed to show a world-space position if there are transformed parent prims.

8. Review questions — active recall

Try before looking at the answers in the following section.

1. A cube at (1, 4) moves 3 units right and 2 units down. Where is it?
2. A point at (2, 0, 1) moves up 0.5 unit. Give its new position.
3. The robot is at world (4, 1), and the cube is at world (9, 6). What is cube position relative to robot if axes align?
4. Why might two robots describe the same stationary object using different coordinates?
5. When can you *not* use coordinate subtraction alone to convert world coordinates to a robot frame?

Review answers

1. (4, 2). 2. (2, 0, 1.5). 3. (5, 5). 4. Different origins and/or axis orientations. 5. When frame axes are rotated relative to one another (a rotation transformation is then required).

9. Learning record and next boundary

Demonstrated in conversation: Correct 2D and 3D coordinate arithmetic; correct aligned-frame relative-position answers; recognition and correction of an inconsistent robot-frame diagram; learner confirmation of Isaac Sim cube movement and USD save.

Not yet independently established: Contents of saved USD file; numerical verification extracted from the stage; performance with rotated frames; robust camera-to-robot calibration.

Follow-up (not begun): Lesson 3 may introduce deeper frame transformations, but no Lesson 3 content is included in this manual.